Engineering & Kaggle-Ops Lessons (Handoff)

Kinh nghiệm code / vận hành positive + negative mà phiên này tìm ra. Dùng cho khung chat mới để tránh lặp lỗi và chạy nhanh hơn. Scripts tham chiếu trong `automation/`.

---

1. Kaggle GPU sessions — các bài học QUAN TRỌNG NHẤT (nhiều cái học bằng lỗi)

1.1 Ngắt (cancel) một GPU session

- **KHÔNG có** lệnh cancel/stop/abort trong Kaggle token API (kiểm tra `dir(kaggle.api)`: chỉ có `kernels_delete`, không có cancel).
- **Push một version mới KHÔNG ngắt session đang queued/running** của kernel đó. (Đây là lỗi tôi mắc; user chỉ ra.) Cố push đè khi đã có session cũ → lỗi "Maximum batch GPU session count of 2 reached".
- **Cách ngắt THẬT (đã verify):** `kaggle kernels delete -y <owner>/<slug>` → xóa kernel, chấm dứt run, giải phóng GPU slot. Push lại sau đó sẽ tạo lại kernel mới. (Destructive với lịch sử version, nhưng không quan trọng.)
- **Push bản CPU no-op (enable_gpu=False) KHÔNG đáng tin để ngắt** GPU session cũ (tôi đã thử, không hiệu quả như mong đợi). Dùng `delete`.

1.2 Giới hạn & concurrency

- Trần là **2 batch GPU session đồng thời** (chính lỗi "max 2" là bằng chứng trần ≥2). **Chạy song song 2 kernel GPU là ĐƯỢC** — đã verify trực tiếp: NB13 + NB14 cùng chạy trên T4 (13:51–14:05).
- **Tránh tích lũy session:** mỗi lần push retry mà không `delete` cái cũ sẽ ăn dần vào trần 2 → kẹt. Luôn `delete` trước khi push lại cho cùng kernel nếu cái cũ còn đang chạy/queued.

1.3 "0 output" KHÔNG đồng nghĩa với treo

- `kaggle kernels output <id>` và `kaggle kernels logs <id>` **trả log rất trễ** — thường chỉ lộ log khi kernel **gần xong / xong**. Một kernel RUNNING với "0 stdout qua API" có thể vẫn đang chạy bình thường (startup, mount dataset lớn). → **Đừng kết luận treo chỉ vì output API trống.**
- Bằng chứng tin cậy hơn về "đã được cấp GPU": chờ dòng output đầu tiên xuất hiện (lag), hoặc đợi tới khi COMPLETE.

1.4 T4 vs RTX PRO 6000 (thực tế phiên này)

- **RTX PRO 6000 có thể BẬN thật** → kernel queued **21 phút, 0 output** dù slot đã trống. Re-queue cũng vào đúng GPU bận. Khi RTX bận, **retry không giúp**; cân nhắc T4.
- **T4 instant, không hàng chờ**, và **chạy song song 2 kernel** → tổng thể thường **chắc + nhanh hơn** cho dữ liệu cỡ vài chục nghìn ảnh.
- **Phần sequential filter là CPU-bound (numpy)** → RTX KHÔNG tăng tốc phần này; RTX chỉ nhanh ở extraction (SALAD forward).
- **Throughput đo thực tế trên T4:** SALAD ~**16 ảnh/s** (NB13 ~12k ảnh extract trong ~12 phút); có lúc ~7–8 ảnh/s khi T4 tải nặng. → ước lượng: ~10k ảnh ≈ 10–25 phút trên T4.

1.5 Stall-rule runner (chính sách user)

- Chính sách user cho RTX: **15 phút output=0 → ngắt session cũ (delete) → tạo session mới; tối đa 4 lần; vẫn fail thì kiểm tra lỗi.** Đã code đúng trong `automation/run_rtx2.py` (`free_and_restart()` dùng `kaggle kernels delete -y` rồi push lại).
- Với T4: không cần restart (instant). Dùng `automation/run_stall.py` (push + poll + decode), đặt `STALL` đủ lớn (vd 1200–1500s) vì extraction lớn chỉ in output lần đầu sau vài phút.

---

2. Quy trình notebook (build-from-src) — hoạt động tốt

- **Nguồn cell trong `nbX_src/*.src`**, marker `#%%CODE` / `#%%MARKDOWN`, ghép theo tên file. Build bằng `automation/build_nb.py <out.ipynb> <src_dir>` → **compile-check mọi code cell** + validate nbformat. Rất hiệu quả để bắt lỗi cú pháp trước khi push.
- **Tái dùng block:** copy `10_p0.src` (config/paths/SALAD loader), `20_lib.src` (metrics/filter/ccbf), `30_extract.src` (extraction), `40_*_load.src` (loader dataset) giữa các notebook → nhất quán, ít bug.

- **Test logic local trước khi push:** exec các cell numpy (lib L.1–L.3 + cell thí nghiệm) trên **synthetic data** đã bắt được nhiều bug (degenerate calibrator khi acc=1.0, unpack sai, off-map labels...). Luôn làm trước mỗi push — tiết kiệm nhiều vòng Kaggle.
- **Lưu ý:** synthetic thường **quá sạch** (softmax đã calibrated) → không thay thế được dữ liệu thật để đánh giá khoa học; chỉ để validate code path.

3. Loaders dataset — gotchas đã xử lý (quan trọng cho notebook mới)

- **RobotCar `robotcar_v2_train.txt`:** dòng = `"<cond>/<cam>/<ts>.jpg"` + **4×4 pose phẳng (12 floats, row-major)**. **Translation = flat index [3,7,11]** (KHÔNG phải `nums[4:7]` — bug cũ của NB2). Ảnh ở `images/<cond>/<cond>/<cam>/...`; resolve robust bằng **index theo basename** (walk mount 1 lần). File chỉ chứa **query conditions** (~200 ảnh/điều kiện); `overcast-reference` KHÔNG có trong train.txt (pose ở NVM/`.db`).
- **CMU `camera-poses/*.txt`:** dòng = `"img_XXXXX_cN_<ts>us.jpg" qw qx qy qz tx ty tz` → **xyz = 3 float cuối** (`nums[-3:]`). Mỗi slice ở `sliceN/sliceN/` (lồng!) với `database/`, `query/`, `camera-poses/` (≈11–12 file pose theo **ngày = điều kiện**). Ảnh query ở `query/`. **BUG đã gặp:** glob `**/slice*` bắt cả `sliceN` lẫn `sliceN/sliceN` → **trùng slice**; phải **dedup theo base dir**. Dùng **1 camera (c0)** để tránh lệch viewpoint.
- **Nordland:** 4 mùa frame-aligned 1:1 (filename `images-NNNNN.png`). GT place = frame-id identity (`gt_match_index`). Full 143k/mùa (35,768 mỗi mùa); subsample bằng `CONFIG.SMOKE_MAX_FRAMES`. **Path finder của 10_p0** (`_find_dir(["data"],["data","annotations"])`) có thể **false-match** robotcar images dir (chứa "overcast-summer"/"winter") nếu attach cùng lúc — chú ý khi attach nhiều dataset.
- **Pose-GT regime matters:** ở tol lỏng (5m) RobotCar/CMU gần như bão hòa → **sweep tolerance** {0.5,1,2,5}m để có regime nhiều lỗi (reliability mới có ý nghĩa). Headline chọn tol có acc ≈ 0.8.

4. SALAD loader (đã verify, dùng lại nguyên)

- Load trực tiếp qua `helper.get_backbone("dinov2_vitb14", {...})` + `helper.get_aggregator("SALAD", {...})`, bypass VPRModel/Lightning; ckpt `bundle/checkpoints/dino_salad.ckpt`. Forward `aggregator(backbone(x))` → dim **8448**, L2-normalized. (Chi tiết config trong `nb*_src/10_p0.src _try_salad`.)
- DINOv2 CLS+mean (dim 1536) là fallback; ép dùng bằng cách để loop extractor chỉ `[("dinov2", _try_dinov2)]`.

5. Metrics / numerics — đã kiểm chứng

- **ECE không đủ** (bị game bởi base-rate khi acc đồng đều) → luôn đi kèm **AURC (risk–coverage)** và **conf-AUROC** (phân biệt correct/incorrect).
- **Logistic calibrator (`fit_logistic`)** degenerate khi fit-set có acc≈1.0 (không có lỗi để học) → fit trên tập **pooled gồm cả điều kiện khó** (cross-fit 50/50) thay vì chỉ benign.
- **Conformal:** dùng **Hoeffding UCB** (RCPS-style) để realized risk ≤ target thật sự finite-sample; prefix-risk thuần sẽ overshoot.
- numpy 2.x: `float(arr)` lỗi với size-1 ndim≥1 → `float(arr.reshape(()))`. `"X" in globals()` (không `dir()`).

6. Môi trường cloud session này (khung chat mới cần biết)

- **Không có** Kaggle token sẵn trong container cloud; phiên này user cấp token. Cài deps: `pip install kaggle nbformat numpy python-dateutil python-slugify tqdm protobuf matplotlib`. Token: `mkdir -p ~/.kaggle && echo <KGAT_...> > ~/.kaggle/access_token && chmod 600 ...` (Kaggle CLI 2.2.5 đọc file này; cũng đọc env `KAGGLE_API_TOKEN`).
- **md→PDF:** `pip install markdown xhtml2pdf`, rồi `python automation/md2pdf.py <in.md> <out.pdf>` (pure-python, không cần system deps; `reportlab` cũng có sẵn).
- Outbound HTTPS qua agent proxy (CA bundle `/root/.ccr/ca-bundle.crt`); `gh`/`gcloud`/`gsutil`/`bq` preconfigured.
- GitHub: push lên branch `claude/elegant-brown-xvnecd` (đã dùng xuyên suốt). Dùng MCP `mcp__github__*` nếu cần PR.

7. Những gì NÊN làm khác lần sau (negative lessons)

1. **Luôn `delete` session RTX cũ trước khi push lại** (đừng dựa vào push-đè hay CPU-noop).

2. **Đừng kết luận kernel treo từ "0 output"** — chờ đủ lâu hoặc tới terminal state.
3. **Ưu tiên T4 cho dữ liệu vừa** (≤ vài chục nghìn ảnh): instant + song song 2 kernel, không rủi ro hàng chờ RTX. Chỉ dùng RTX khi thật sự cần (rất nhiều ảnh) và kiểm tra GPU rảnh trước.
4. **Test local synthetic trước mọi push** — đã cứu nhiều vòng.
5. **Dedup khi glob cấu trúc lồng** (CMU `sliceN/sliceN`).
6. **Chạy một thí nghiệm quyết định mỗi lần** (không fan-out nhiều run khi kết quả một run định hướng các run sau).